

Task 05: Networking and Security Operations with SIEM, Forensics, and Traffic Analysis

Prepared by: Kalash Mahajan

Date: 30th January 2026

1. Introduction

This report documents the practical execution of Task 05: Networking and Security Operations with SIEM, Forensics, and Traffic Analysis, performed in a controlled virtual lab environment.

The objective of this task was to bridge network-level visibility with SOC-style monitoring and incident analysis by combining packet-level inspection using Wireshark with centralized log monitoring using the ELK Stack (Elasticsearch, Logstash, Kibana).

The scope of work included:

- Capturing and analyzing real network traffic generated from a Linux virtual machine
- Identifying active protocols, top talkers, and communication patterns
- Deploying and validating an ELK-based SIEM setup on Ubuntu
- Ingesting system logs via Logstash and visualizing them in Kibana
- Simulating an unauthorized SSH access attempt from a separate Windows host
- Detecting and validating the incident using both system logs and SIEM queries

All observations, findings, and conclusions in this report are derived directly from hands-on experimentation, command-line execution, and captured evidence. Screenshots and logs are included at each stage to demonstrate alignment with real-world networking and SOC workflows rather than theoretical scenarios.

2. Objectives

The objectives of Task 05 were defined to bridge networking fundamentals with SOC-level security operations, using a hands-on lab-based approach. Each objective below directly maps to tasks performed during this exercise.

- Design and validate a logical subnet suitable for a small office environment, ensuring efficient IP utilization and scalability.
- Capture and analyze real network traffic using Wireshark to understand protocol behavior and communication patterns.
- Identify and troubleshoot network-level issues, including DNS resolution behavior and ICMP/TCP interactions.
- Deploy and configure a SIEM solution using the ELK Stack (Elasticsearch, Logstash, Kibana) on a Linux environment.
- Ingest and visualize system and authentication logs to enable centralized monitoring.
- Simulate a security incident (failed SSH authentication attempts) and perform log-based forensic analysis.
- Correlate network activity with SIEM data to demonstrate incident detection and investigation workflows.
- Perform basic threat hunting by querying logs for indicators such as repeated failed logins and suspicious access attempts.

3. Network Subnet Design

3.1 Requirements

The subnet was designed to represent a small office network with the following requirements:

- Approximately 20 end devices, including:
 - User workstations
 - Printers
 - Servers / network services
- Single LAN segment with internal communication
- Private IP addressing (RFC 1918)
- Simple structure suitable for monitoring and traffic analysis

- Future scalability without immediate reconfiguration

3.2 IP Addressing and CIDR Selection

- **Chosen IP range:** 192.168.52.0/24
- **Address type:** Private IPv4 address space
- **CIDR notation:** /24

Justification:

- Supports up to 254 usable IP addresses, which comfortably exceeds the current requirement of 20 devices.
- Commonly used in small enterprise and lab environments.
- Simplifies routing, monitoring, and troubleshooting during packet capture and SIEM analysis.
- Matches the actual lab environment used during traffic capture and ELK deployment.

3.3 Subnet Calculations

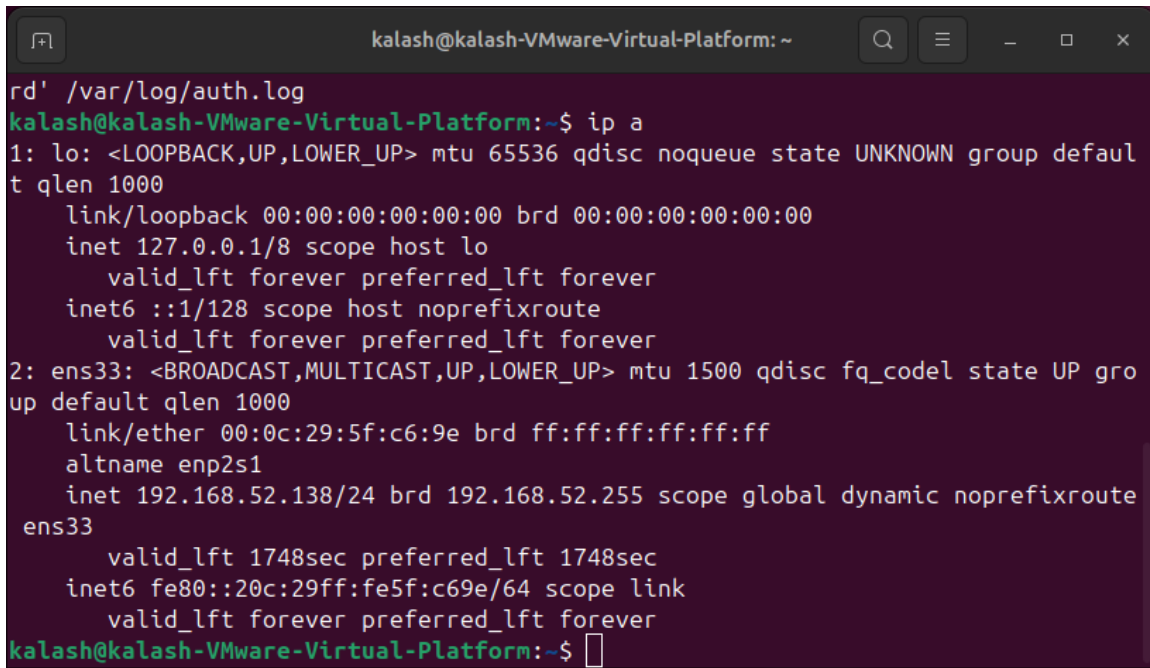
Parameter	Value
Network Address	192.168.52.0
Subnet Mask	255.255.255.0
CIDR	/24
Total IP Addresses	256
Usable IP Addresses	254
Network Address	192.168.52.0
Broadcast Address	192.168.52.255
Usable Host Range	192.168.52.1 – 192.168.52.254

This configuration ensures sufficient address space while keeping the network simple and manageable.

3.4 Network Diagram / Simulation

- The subnet was implemented directly within a virtualized lab environment.
- Systems were connected within the same /24 network segment.
- Network behavior was validated using:
 - ip a for interface verification

- Live traffic capture using Wireshark

A terminal window titled 'kalash@kalash-VMware-Virtual-Platform: ~' showing the output of the 'ip a' command. The output displays details for the loopback interface 'lo' (127.0.0.1) and the ethernet interface 'ens33' (192.168.52.138). The terminal text is as follows:

```
rd' /var/log/auth.log
kalash@kalash-VMware-Virtual-Platform:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:5f:c6:9e brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.52.138/24 brd 192.168.52.255 scope global dynamic noprefixroute ens33
        valid_lft 1748sec preferred_lft 1748sec
    inet6 fe80::20c:29ff:fe5f:c69e/64 scope link
        valid_lft forever preferred_lft forever
kalash@kalash-VMware-Virtual-Platform:~$
```

Figure 1: IP configuration of the Ubuntu system showing 192.168.52.0/24 subnet

4. Network Traffic Analysis

4.1 Environment Setup

Network traffic analysis was performed using a Linux-based virtual machine configured within a private subnet. The system was actively connected to the network and generating normal user-driven traffic.

Wireshark was used as the primary packet capture and analysis tool to observe live network activity at the packet level.

4.2 Traffic Capture

A live packet capture was conducted for approximately 10 minutes during normal system usage. The capture scope included all inbound and outbound traffic on the active network interface.

The captured data represents a baseline view of typical network behavior within a small office-style environment.

4.3 Protocol Analysis

Analysis of the captured packets showed the presence of multiple network protocols commonly observed in everyday network communication:

- **TCP** traffic formed the majority of the capture, primarily carrying encrypted application data.
- **DNS** traffic was observed during domain name resolution for external services.
- **ICMP** packets were generated during connectivity testing.
- **ARP** traffic was present within the local subnet for address resolution.
- **TLS** traffic confirmed the use of secure HTTPS connections.

The protocol distribution indicated normal and expected network behavior.

4.4 Traffic Patterns

Conversation-level analysis revealed that the local host generated most of the observed traffic. The system communicated frequently with external servers for DNS resolution and secure web access.

Observed traffic patterns included:

- Short-lived TCP connections
- Regular DNS queries
- Consistent traffic volume with no abnormal spikes

No suspicious communication patterns were detected during this phase.

4.5 Observations and Findings

- The captured traffic consisted primarily of legitimate DNS and HTTPS activity.
- All observed protocols and destinations aligned with expected user behavior.
- This analysis established a baseline network profile used for later incident investigation.

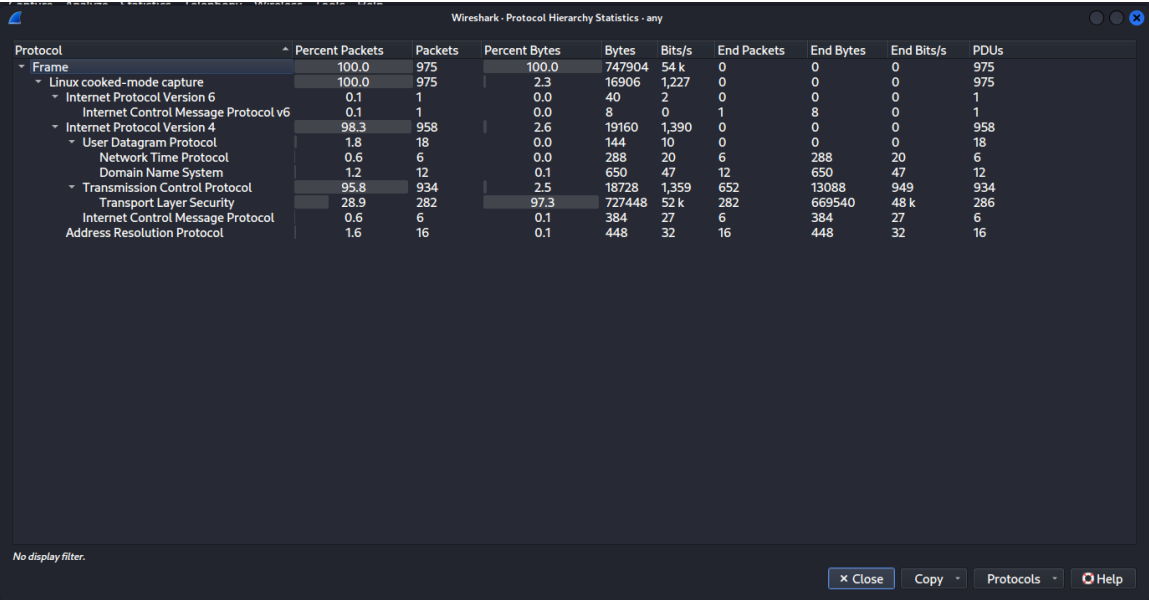


Figure 2: Protocol Hierarchy Statistics

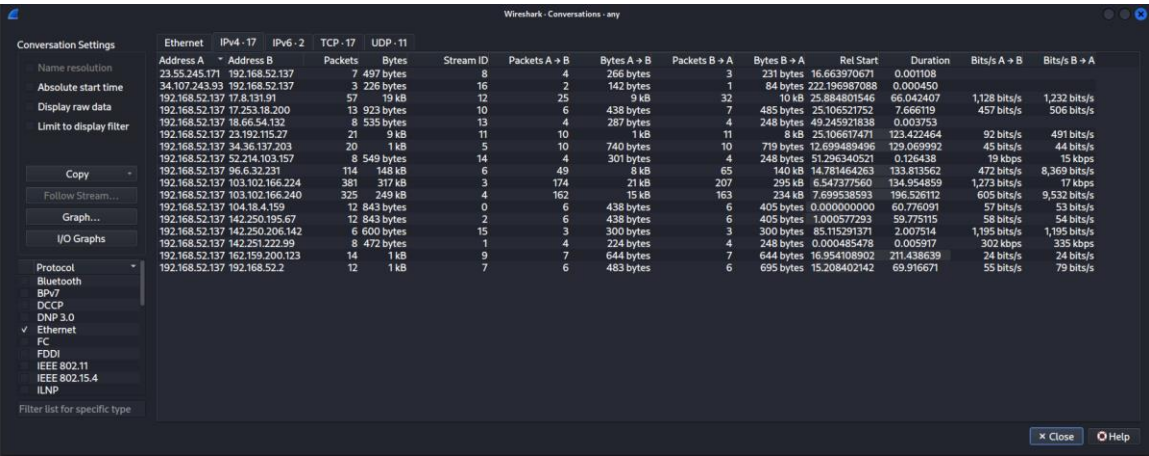


Figure 3: Conversations View

No.	Time	Source	Destination	Protocol	Length	Info
941	85.105818848	192.168.52.137	192.168.52.2	DNS	72	Standard query 0x35e6 A google.com
942	85.105871137	192.168.52.137	192.168.52.2	DNS	72	Standard query 0x1fe7 AAAA google.com
943	85.107641037	Vmware-c0:00:08		ARP	62	Who has 192.168.52.2? Tell 192.168.52.1
944	85.113984233	192.168.52.2	192.168.52.137	DNS	88	Standard query response 0x35e6 A google.com A 142.250.206.142
945	85.113984415	192.168.52.2	192.168.52.137	DNS	180	Standard query response 0x1fe7 AAAA google.com AAAA 2404:6808:4009:020::200e
946	85.115291371	192.168.52.137	142.250.206.142	ICMP	100	Echo (ping) request id=0x0002, seq=1/256, ttl=64 (reply in 947)
947	85.115865438	142.250.206.142	192.168.52.137	ICMP	100	Echo (ping) reply id=0x0002, seq=1/256, ttl=128 (request in 946)
948	85.119927921	192.168.52.137	192.168.52.2	DNS	90	Standard query 0x015c PTR 142.206.250.142.in-addr.arpa
949	85.125073379	192.168.52.2	192.168.52.137	DNS	160	Standard query response 0x015c PTR 142.206.250.142.in-addr.arpa PTR lcboma-aq-in-f14.1e100.net PTR del11s21-
950	86.116619336	192.168.52.137	142.250.206.142	ICMP	100	Echo (ping) request id=0x0002, seq=2/512, ttl=64 (reply in 951)
951	86.123015115	142.250.206.142	192.168.52.137	ICMP	100	Echo (ping) reply id=0x0002, seq=2/512, ttl=128 (request in 950)
952	87.110886363	192.168.52.137	142.250.206.142	ICMP	100	Echo (ping) request id=0x0002, seq=3/768, ttl=64 (reply in 953)
953	87.122805713	142.250.206.142	192.168.52.137	ICMP	100	Echo (ping) reply id=0x0002, seq=3/768, ttl=128 (request in 952)
954	88.130945445	192.168.52.137	162.159.200.123	NTP	92	NTP Version 4, client
955	88.137381034	162.159.200.123	192.168.52.137	NTP	92	NTP Version 4, server
956	91.926580987	17.8.131.91	192.168.52.137	TLShv.3	119	Application Data, Application Data
957	91.926841100	192.168.52.137	17.8.131.91	TLShv.3	95	Application Data
958	91.927059428	192.168.52.137	17.8.131.91	TLShv.3	89	Application Data
959	91.927082953	17.8.131.91	192.168.52.137	TCP	62	443 - 51650 [ACK] Seq=8272 Ack=7871 Win=64240 Len=0
960	91.927208142	17.8.131.91	192.168.52.137	TCP	62	443 - 51650 [ACK] Seq=8272 Ack=7896 Win=64239 Len=0
961	92.285922233	Vmware-c0:00:08		ARP	62	Who has 192.168.52.2? Tell 192.168.52.1
962	93.085597246	Vmware-c0:00:08		ARP	62	Who has 192.168.52.2? Tell 192.168.52.1
963	94.015947242	Vmware-c0:00:08		ARP	62	Who has 192.168.52.2? Tell 192.168.52.1
964	94.289486795	192.168.52.137	23.192.115.27	TLShv.2	95	Application Data
965	94.289491365	192.168.52.137	23.192.115.27	TLShv.2	95	Application Data
966	94.289645854	96.6.32.231	192.168.52.137	TCP	62	443 - 44512 [ACK] Seq=136067 Ack=5039 Win=64240 Len=0
967	94.289646114	23.192.115.27	192.168.52.137	TCP	62	443 - 59366 [ACK] Seq=6871 Ack=801 Win=64240 Len=0
968	94.293407946	96.6.32.231	192.168.52.137	TLShv.2	95	Application Data
969	94.293408271	23.192.115.27	192.168.52.137	TLShv.2	95	Application Data
970	94.293448928	192.168.52.137	23.192.115.27	TCP	56	59366 - 443 [ACK] Seq=801 Ack=8910 Win=65535 Len=0
971	94.335480979	192.168.52.137	96.6.32.231	TCP	56	44512 - 443 [ACK] Seq=9039 Ack=136106 Win=65535 Len=0
972	95.080592564	Vmware-c0:00:08		ARP	62	Who has 192.168.52.2? Tell 192.168.52.1
973	95.875851019	Vmware-c0:00:08		ARP	62	Who has 192.168.52.2? Tell 192.168.52.1
974	96.810549782	Vmware-c0:00:08		ARP	62	Who has 192.168.52.2? Tell 192.168.52.1

Figure 4: Packet List View

5. Network Protocol Troubleshooting

5.1 Issue Description

As part of the troubleshooting task, a DNS misconfiguration was intentionally introduced in the Linux virtual machine to simulate a common real-world network issue.

After modifying DNS settings:

- External websites could not be reached
- Name resolution failed while IP-level connectivity was still available

This scenario was used to observe protocol behavior, traffic patterns, and system logging during a DNS failure condition.

5.2 Troubleshooting Methodology

A step-by-step troubleshooting process was followed:

- Network connectivity verification was performed to confirm that the system still had a valid IP address and gateway.
- DNS behavior analysis was conducted by attempting to access websites and observing resolution failures.

- Packet-level inspection was performed using Wireshark to identify failed DNS queries.
- System log analysis was carried out to confirm that DNS-related and network activity logs were being generated.

This approach helped isolate the issue specifically to the DNS layer rather than general network connectivity.

5.3 Resolution

The issue was resolved by restoring correct DNS configuration.

Once DNS settings were corrected:

- Website access was successfully restored
- DNS queries completed normally
- Network traffic returned to expected patterns

The generated logs and packet captures during the failure window were preserved for further analysis and correlation in the SIEM platform.

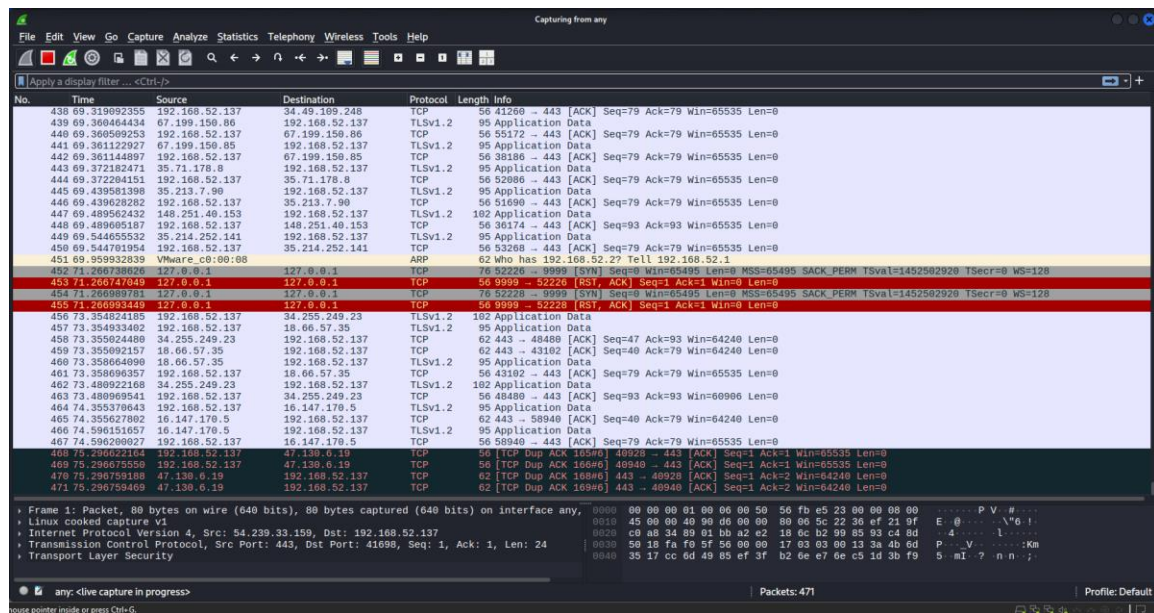


Figure 5: Packet captures during the failure

6. SIEM Setup (ELK Stack)

6.1 Architecture Overview

A centralized SIEM lab was implemented using the ELK Stack (Elasticsearch, Logstash, Kibana) on an Ubuntu Linux virtual machine.

The setup follows a single-node SIEM architecture, suitable for SOC learning and incident investigation:

- **Ubuntu Linux VM** – Log source and SIEM host
- **Logstash** – Ingests and parses system and authentication logs
- **Elasticsearch** – Stores, indexes, and enables fast search on logs
- **Kibana** – Used for visualization, filtering, and forensic analysis

This architecture allowed end-to-end visibility from log generation to incident investigation within a controlled lab environment.

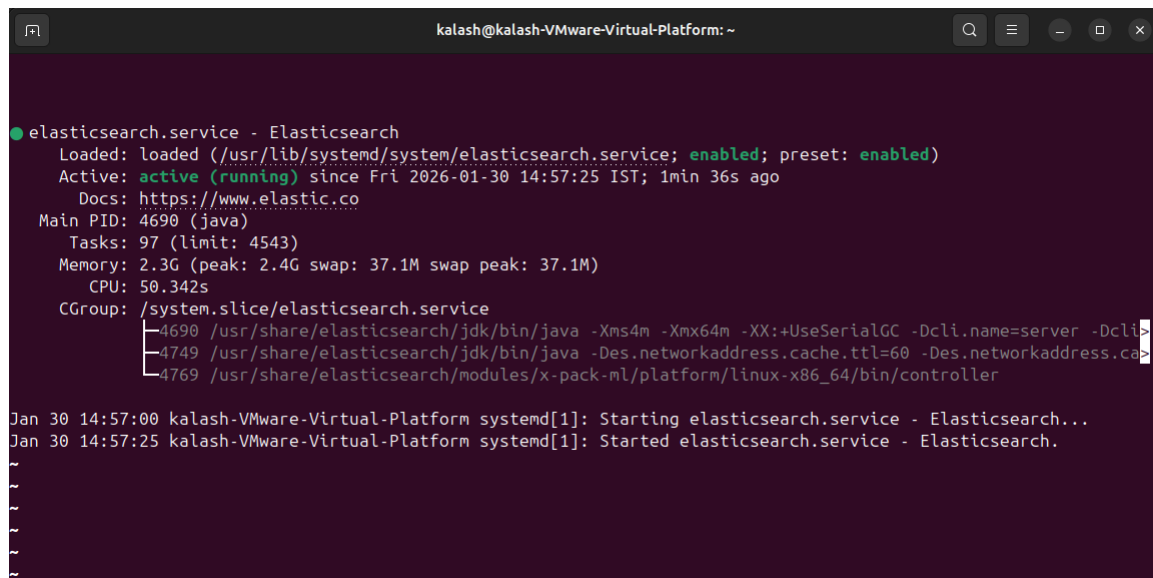
6.2 Elasticsearch Setup

Elasticsearch was installed as the core data storage and search engine.

Actions performed:

- Installed Elasticsearch and enabled it as a system service
- Verified successful startup and cluster health using REST API calls
- Confirmed secure access using authenticated requests
- Validated index creation for ingested log data

Elasticsearch successfully indexed log events generated from system activity and simulated incidents.

A terminal window titled 'kalash@kalash-VMware-Virtual-Platform: ~' displays the status of the 'elasticsearch.service'. The service is shown as 'loaded' and 'enabled', with an 'Active' status of 'active (running)' since 'Fri 2026-01-30 14:57:25 IST; 1min 36s ago'. It lists the main PID as 4690 (java), tasks as 97, memory usage as 2.3G, and CPU usage as 50.342s. The CGroup path is '/system.slice/elasticsearch.service'. Below this, a tree view shows the process hierarchy: PID 4690 is the parent, with children PID 4749 (java) and PID 4769 (controller). At the bottom, system logs show the service starting at 'Jan 30 14:57:00' and starting successfully at 'Jan 30 14:57:25'.

```
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-01-30 14:57:25 IST; 1min 36s ago
     Docs: https://www.elastic.co
    Main PID: 4690 (java)
      Tasks: 97 (limit: 4543)
     Memory: 2.3G (peak: 2.4G swap: 37.1M swap peak: 37.1M)
        CPU: 50.342s
    CGroup: /system.slice/elasticsearch.service
            └─4690 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli
                  └─4749 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.ca
                      └─4769 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

Jan 30 14:57:00 kalash-VMware-Virtual-Platform systemd[1]: Starting elasticsearch.service - Elasticsearch...
Jan 30 14:57:25 kalash-VMware-Virtual-Platform systemd[1]: Started elasticsearch.service - Elasticsearch.
~
~
~
~
~
```

Figure 6: elasticsearch active

```
kalash@kalash-VMware-Virtual-Platform: ~  
Password for the [elastic] user successfully reset.  
New value: T+j8*8W7_XVUkPHEezs5  
kalash@kalash-VMware-Virtual-Platform:~$ ^C  
kalash@kalash-VMware-Virtual-Platform:~$ curl -k -u elastic https://localhost:9200  
Enter host password for user 'elastic':  
{  
  "name" : "kalash-VMware-Virtual-Platform",  
  "cluster_name" : "elasticsearch",  
  "cluster_uuid" : "AjBY0oP2QGWLdJSpjhJsaw",  
  "version" : {  
    "number" : "8.19.10",  
    "build_flavor" : "default",  
    "build_type" : "deb",  
    "build_hash" : "493241b351be6d9f40d52a1406c91a23b4148821",  
    "build_date" : "2026-01-08T22:07:49.939644068Z",  
    "build_snapshot" : false,  
    "lucene_version" : "9.12.2",  
    "minimum_wire_compatibility_version" : "7.17.0",  
    "minimum_index_compatibility_version" : "7.0.0"  
  },  
  "tagline" : "You Know, for Search"  
}
```

Figure 7: elasticsearch log file

6.3 Logstash Configuration

Logstash was configured to act as the log ingestion and processing layer.

Configuration details:

- Logstash was set to monitor local Linux system logs
- Logs were parsed and forwarded to Elasticsearch securely
- Custom index naming was applied to organize logs by date
- Log ingestion was verified using test log entries generated via the system logger

This ensured real-time ingestion of authentication failures, system events, and network-related logs.

```
kalash@kalash-VMware-Virtual-Platform: ~  
` is configured with `pipeline.ecs_compatibility: v8` setting. All plugins in th  
is pipeline will default to `ecs_compatibility => v8` unless explicitly configur  
ed otherwise.  
Configuration OK  
[INFO ] 2026-01-30 15:28:27.323 [LogStash::Runner] runner - Using config.test_an  
d_exit mode. Config Validation Result: OK. Exiting Logstash  
kalash@kalash-VMware-Virtual-Platform:~$ sudo systemctl enable logstash  
Created symlink /etc/systemd/system/multi-user.target.wants/logstash.service → /  
usr/lib/systemd/system/logstash.service.  
kalash@kalash-VMware-Virtual-Platform:~$ sudo systemctl start logstash  
kalash@kalash-VMware-Virtual-Platform:~$ sudo systemctl status logstash  
● logstash.service - logstash  
   Loaded: loaded (/usr/lib/systemd/system/logstash.service; enabled; preset:~  
   Active: active (running) since Fri 2026-01-30 15:30:18 IST; 6s ago  
   Main PID: 7631 (java)  
     Tasks: 21 (limit: 4543)  
    Memory: 223.3M (peak: 223.8M)  
       CPU: 9.068s  
    CGroup: /system.slice/logstash.service  
            └─7631 /usr/share/logstash/jdk/bin/java -Xms1g -Xmx1g -Djava.awt.h>  
Jan 30 15:30:18 kalash-VMware-Virtual-Platform systemd[1]: Started logstash.ser>  
Jan 30 15:30:18 kalash-VMware-Virtual-Platform logstash[7631]: Using bundled JD>  
lines 1-12/12 (END)
```

Figure 8: logstash status verification

6.4 Kibana Setup and Log Analysis

Kibana was configured as the visualization and investigation interface.

Steps performed:

- Connected Kibana to the Elasticsearch backend
- Created a data view for system log indices
- Used the Discover feature to validate log ingestion
- Applied filters to identify security-relevant events such as failed SSH login attempts
- Adjusted time ranges to correlate logs with simulated attack activity

Kibana enabled efficient investigation of incidents and supported timeline-based forensic analysis.

```
kalash@kalash-VMware-Virtual-Platform: ~
9/production.html#openssl-legacy-provider
Created Kibana keystore in /etc/kibana/kibana.keystore
kalash@kalash-VMware-Virtual-Platform:~$ sudo systemctl daemon-reload
kalash@kalash-VMware-Virtual-Platform:~$ sudo systemctl enable kibana
Created symlink /etc/systemd/system/multi-user.target.wants/kibana.service → /usr/lib/systemd/system/kibana.service.
kalash@kalash-VMware-Virtual-Platform:~$ sudo systemctl start kibana
kalash@kalash-VMware-Virtual-Platform:~$ sudo systemctl status kibana
● kibana.service - Kibana
   Loaded: loaded (/usr/lib/systemd/system/kibana.service; enabled; preset: e>
   Active: active (running) since Fri 2026-01-30 15:11:03 IST; 6s ago
     Docs: https://www.elastic.co
  Main PID: 5578 (node)
    Tasks: 11 (limit: 4543)
   Memory: 271.7M (peak: 272.0M)
      CPU: 6.732s
   CGroup: /system.slice/kibana.service
           └─5578 /usr/share/kibana/bin/./node/glibc-217/bin/node /usr/share>

Jan 30 15:11:03 kalash-VMware-Virtual-Platform systemd[1]: Started kibana.servi>
Jan 30 15:11:03 kalash-VMware-Virtual-Platform kibana[5578]: Kibana is currentl>
Jan 30 15:11:05 kalash-VMware-Virtual-Platform kibana[5578]: {"log.level":"info>
Jan 30 15:11:05 kalash-VMware-Virtual-Platform kibana[5578]: Native global cons>
Jan 30 15:11:07 kalash-VMware-Virtual-Platform kibana[5578]: [2026-01-30T15:11:>
Jan 30 15:11:07 kalash-VMware-Virtual-Platform kibana[5578]: [2026-01-30T15:11:>
lines 1-17/17 (END)
```

Figure 9: kibana status verification

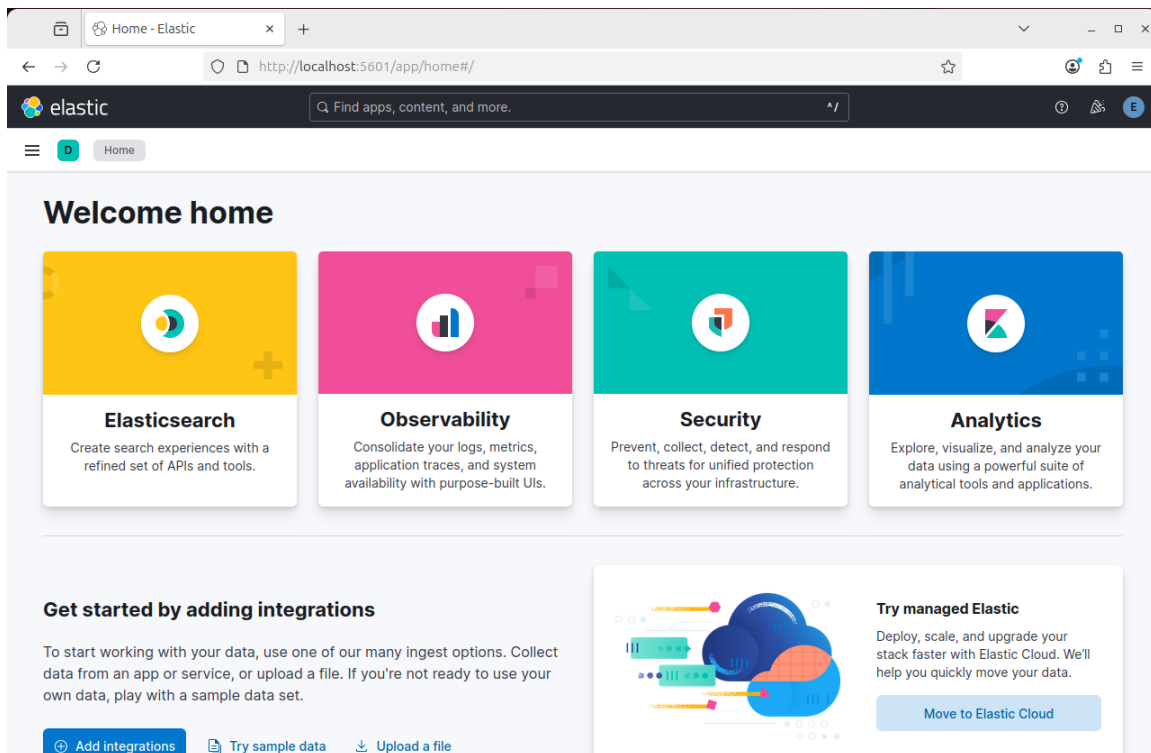


Figure 10: elastic dashboard

7. Incident Simulation and Network Forensics

7.1 Incident Scenario

A simulated security incident was performed by attempting unauthorized SSH access to the Ubuntu Linux system from a local Windows 11 machine.

The incident involved:

- Repeated SSH login attempts using an invalid username
- Intentional use of incorrect credentials
- Goal: generate authentication failures for both network-level and SIEM-level detection

This scenario represents a common real-world attack pattern such as SSH brute-force or credential probing.

7.2 Traffic Capture During Incident

During the incident, network traffic was captured using Wireshark.

Capture details:

- Traffic captured on the active network interface during SSH attempts
- Focus on SSH-related packets over TCP port 22
- Capture included connection attempts, session initiation, and termination packets

The packet capture provided visibility into the network behavior associated with the failed authentication attempts.

7.3 Forensic Analysis

Forensic analysis was performed using both packet-level data and log-level data.

Findings from network traffic:

- SSH connection attempts were observed from the Windows host IP
- Destination port identified as TCP 22
- Repeated connection attempts indicated suspicious behavior
- TCP handshake attempts were visible even though authentication failed

Findings from system logs:

- Authentication failures were recorded for an invalid user
- Source IP address matched the attacking Windows machine
- Timestamps aligned with the Wireshark capture

This confirmed that the observed network traffic directly resulted in logged security events.

7.4 SIEM Correlation and Timeline

Correlation was performed by mapping Wireshark traffic with ELK Stack logs.

Correlation process:

- SSH failure events were identified in Kibana using system log data
- Time range was adjusted to match the incident window
- Source IP, username, and event timestamps were correlated
- Network evidence and SIEM logs confirmed the same attack sequence

This correlation enabled reconstruction of a clear incident timeline, demonstrating how network activity translates into actionable SIEM alerts and forensic evidence.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\HP> ssh fakeuser@192.168.52.138
The authenticity of host '192.168.52.138 (192.168.52.138)' can't be established.
ED25519 key fingerprint is SHA256:6QsfzyRXZToQoZnJY65W0NDy68FAxdtb0kGBFtvStQQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.52.138' (ED25519) to the list of known hosts.
fakeuser@192.168.52.138's password:
Permission denied, please try again.
fakeuser@192.168.52.138's password:
Permission denied, please try again.
fakeuser@192.168.52.138's password:
fakeuser@192.168.52.138: Permission denied (publickey,password).
PS C:\Users\HP>
```

Figure 11: failed logins

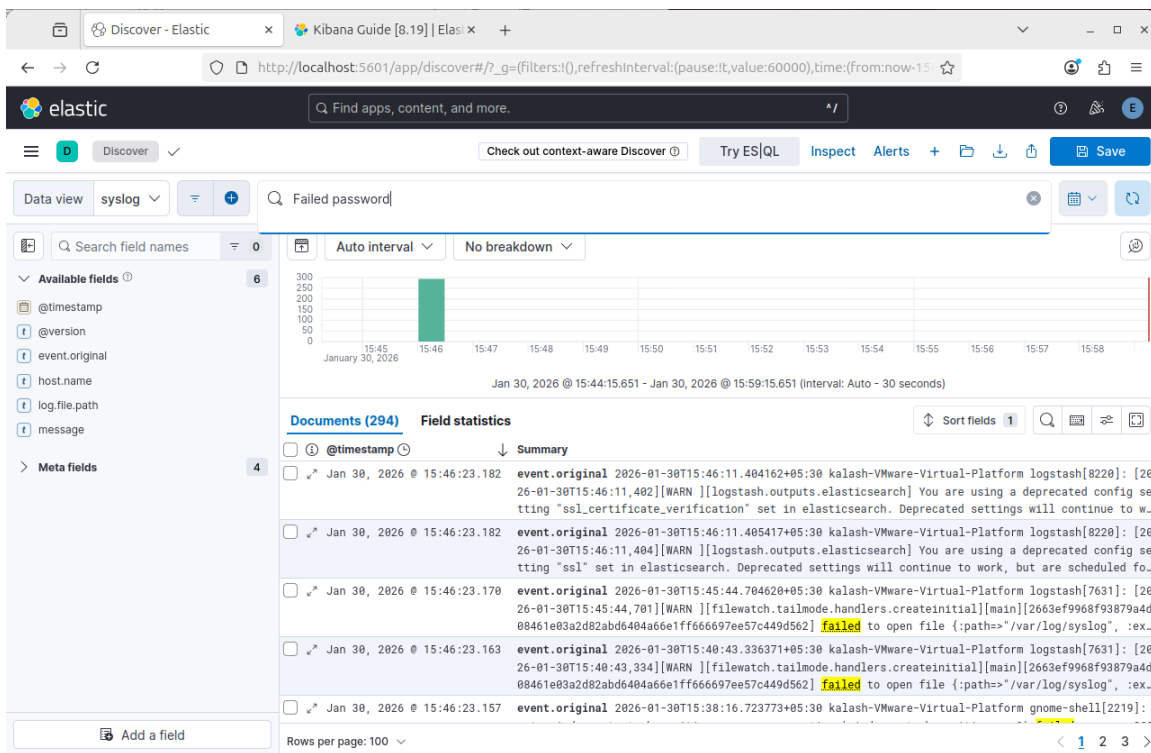


Figure 12: logs generated

8. Threat Hunting

8.1 Threat Hunting Approach

Threat hunting was performed proactively using SIEM log data and system logs ingested into the ELK Stack.

The approach focused on:

- Reviewing authentication-related logs
- Identifying abnormal or repeated access attempts
- Correlating log patterns with known attack behaviors (e.g., brute-force attempts)

Instead of waiting for alerts, logs were manually analyzed in Kibana to identify suspicious activity patterns.

8.2 Indicators of Compromise (IOCs)

During analysis, the following IOCs were identified:

- Repeated failed SSH login attempts
- Use of invalid or non-existent usernames
- Multiple authentication failures from the same source IP
- SSH access attempts targeting TCP port 22
- Short time intervals between repeated login attempts

These indicators align with common credential probing or brute-force attack behavior.

8.3 Findings

The threat hunting activity successfully identified suspicious behavior related to unauthorized SSH access attempts.

Key findings:

- The source IP matched the attacking Windows machine
- Log events were consistent across multiple failed attempts
- No successful authentication was observed, indicating the attack was unsuccessful
- SIEM visibility allowed early detection before escalation

This demonstrates how proactive threat hunting can uncover potential threats even when no critical alert is triggered.

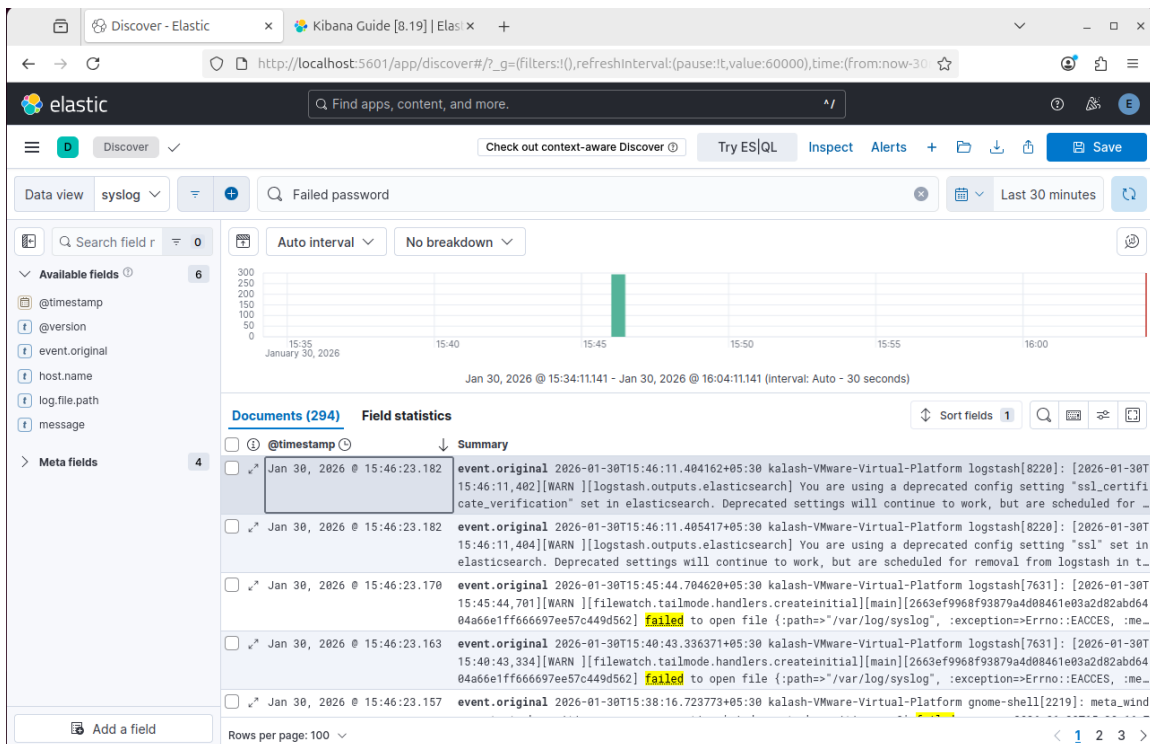


Figure 13: Failed Login Logs

9. Tools Used

The following tools and technologies were used to complete Task 05:

- **Wireshark** – Network traffic capture and packet-level analysis
- **ELK Stack**
 - Elasticsearch – Log storage and indexing
 - Logstash – Log ingestion and parsing (system logs)
 - Kibana – Log analysis, visualization, and threat hunting
- **OpenSSH** – SSH service for access simulation
- **Ubuntu Linux** – SIEM server and log source
- **Windows 11** – Source machine for simulated attack activity
- **Linux system logs (syslog, auth.log)** – Incident and authentication analysis

10. Key Learnings

- Gained hands-on experience in correlating network traffic with SIEM logs to investigate security incidents end-to-end.
- Understood how common attacks like unauthorized SSH access appear at both packet and log levels.
- Learned to troubleshoot configuration issues in ELK and ensure reliable log ingestion and visibility for threat detection.

11. Conclusion

This task provided practical exposure to networking fundamentals, traffic analysis, and SOC operations using a real SIEM setup. By combining subnet design, packet inspection, ELK-based monitoring, and incident simulation, the task demonstrated how security events are detected, analyzed, and validated in real-world environments. The exercise closely reflects day-to-day responsibilities of a SOC analyst handling network-based threats.